Online Perfume Store

In HTML, CSS & JavaScript

Study Course

B204 App & Web Development

By

Chehab Hany Mohamed Elsayed Elsayed Elmenoufi

Student Number: GH1034223

Under the Guidance of

Prof. Alireza Mahmoud

Github URL: https://github.com/Chippo90/App-Web-Development/tree/main

Gisma University of Applied Sciences

Berlin, Germany

March 2026

Table of Contents

1. Introduction *4*

2. System Architecture *4*

2.1 Frontend "Client" 4

2.2 Backend "Server" 4

2.3 Database "Mongo" 5

2.4 Payment Gateway 5

3. Implementation *6*

3.1 Overview 6

3.2 Backend 6

3.2.1 Server Configuration 6

3.2.2 Database Design 7

3.3 Authentication System 8

3.3.1 User Registration 8

3.3.2 Login and JWT 9

3.3.3 Route Protection 9

3.4 Frontend 10

3.4.1 HTML Design 12

3.4.2 CSS Styling 12

3.4.3 JavaScript 13

3.5 PayPal Integration 14

3.6 Docker Integration 15

4. Result Analysis *16*

4.1 Overview 16

4.2 Steps 16

4.3 Page Screeshots 16

4.4 User Authentication 18

4.5 Product Management 20

4.6 Cart Management 20

4.7 PayPal Payment 21

4.8 Summary 21

5. Challenges and Solutions *21*

5.1 Challenge #1 21

5.2 Challenge #2 21

5.3 Challenge #3 22

5.4 Challenge #4 22

5.5 Challenge #5 22

6. Conclusion *22*

7. Future Work *22*
8. References *23*

1. Introduction

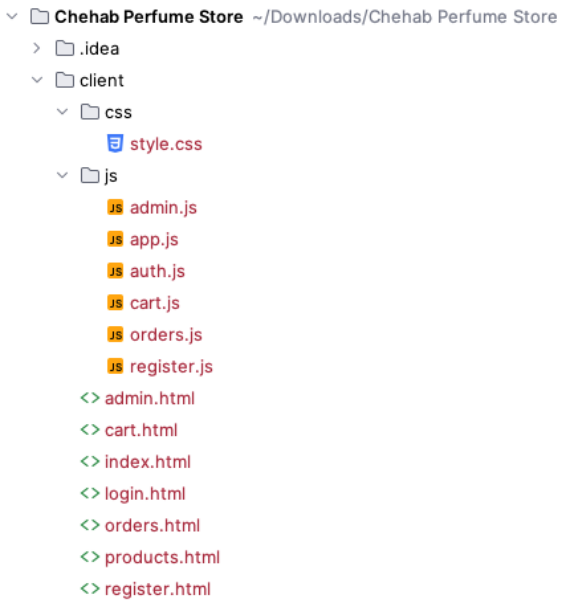
In this project, I designed and implemented a full stack web application for an online perfume store. The store allows customer to read a brief about the company, browse products, add products to the cart and secure checkout through PayPal. Additionally, the store includes authentication which allows admins to manage and add products safely.
The project shows a demonstration of the use of recent technologies including Node.js, Express.js, MongoDB, Docker, JWT authentication and PayPal Sandbox Integration.

2. System Architecture

The system has 4 main structures:-

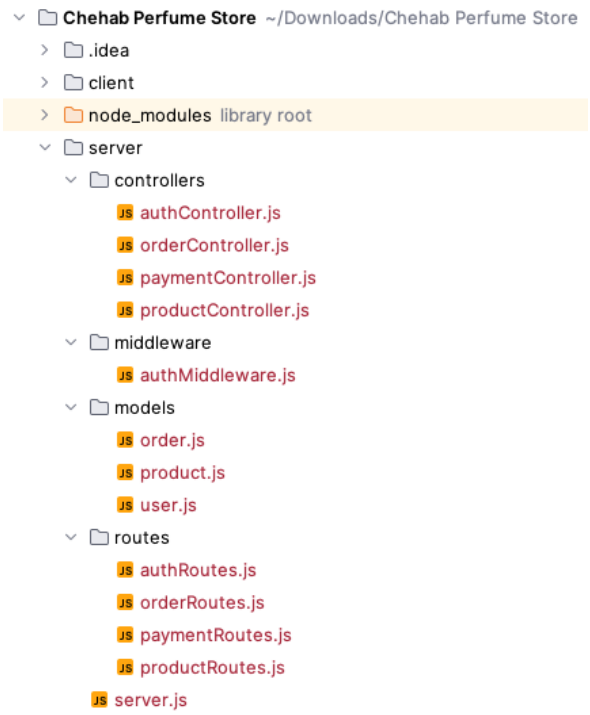
2.1 Frontend "Client"

- I built it with HTML, CSS and JS.
- I handles:
  - Landing page display.
  - Product display.
  - Payment User interface.
  - Authentication.



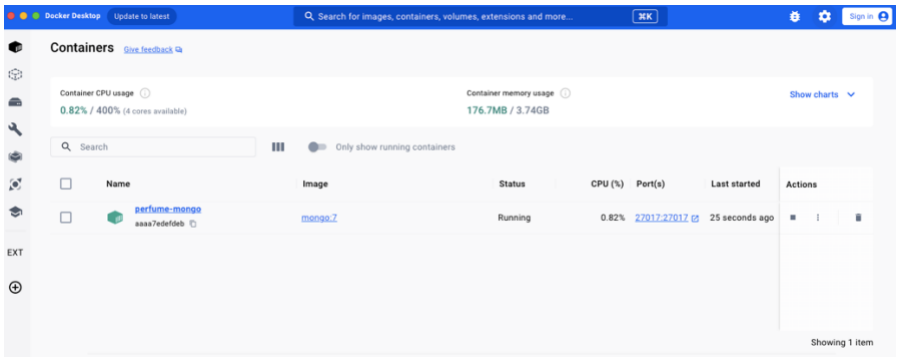
2.2 Backend "Server"

- I built it with Node.js, Express.js and RESTful API.(*Node.js REST API*, no date)
- It handles:
  - Controllers.(*Controllers in ExpressJS*, 14:43:00+00:00)
  - Routes.(*Express routing*, no date)
  - Middleware.(*Middleware in Express*, 00:30:07+00:00)(*Node.js Middleware Concept*, no date)
  - JWT authentication. (*JSON Web Token (JWT)*, 00:00:39+00:00)
  - Admin role authorization for managing products.



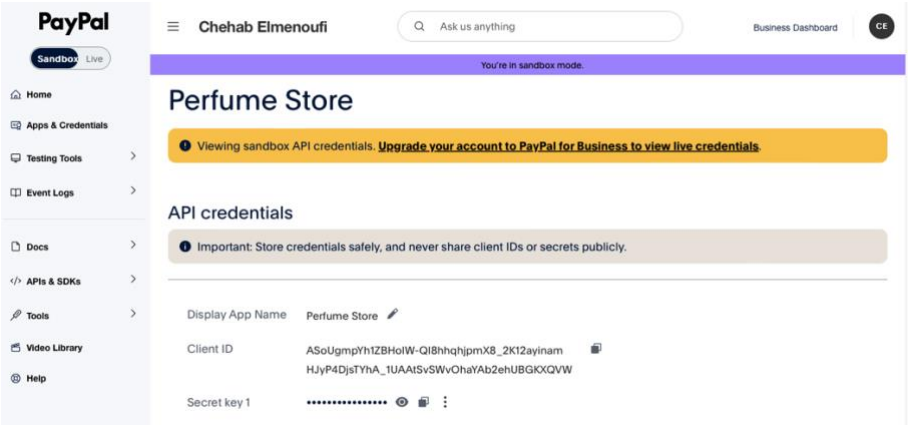
2.3 Database "Mongo"

- MongoDB: it runs inside Docker container.(*Running MongoDB in Docker: A Complete Guide with Examples*, no date)
- Mongoose ODM: it's used for schema modeling.(*Mongoose Tutorial*, 16:52:58+00:00)



2.4 Payment Gateway

- PayPal Sandbox REST API.(*Get Started with PayPal REST APIs*, no date)
- OAuth2 authentication.(Alwis, 2025)



3. Implementation

3.1 Overview

The application was implemented using a full stack architecture. The backend follows Model View Controller pattern while the frontend was developed using HTML, CSS and JS.

3.2 Backend

3.2.1 Server Configuration

The server configuration includes:

- Environment variable loading using .env.(*Node.js — How to read environment variables from Node.js*, no date)

```
//Load environment variables
require("dotenv").config();
```

- JSON body parsing middleware.(*Node.js Middleware Concept*, no date)

```
//Middleware
app.use(express.json());
```

- Static file serving.

```
//Serve frontend
app.use(express.static(path.join(__dirname, "../client")));
```

- API route registration.

```
//Register API routes
app.use("/api/products", productRoutes);
app.use("/api/orders", orderRoutes);
app.use("/api/auth", authRoutes);
app.use("/api/payments", paymentRoutes);
```

- MongoDB connection.

```
//Connect to MongoDB
mongoose.connect(process.env.MONGO_URI) Promise<Mongoose>
    .then(() : void => console.log("MongoDB Connected")) Promise<void>
    .catch(err => console.log(err));
```

The middleware design makes sure that:

- All requests are parsed before reaching controllers.
- Protected routes validate authentication tokens.
- User role access control is enforced.

3.2.2 Database Design

MongoDB was used for NoSQL database and accessed through Mongoose ODM. The database runs inside a docker contatiner.

User Schema:

- Name

```
//User name
name: {
    type: String,
    required: true
},
```

- Email

```
//Email address
email: {
    type: String,
    required: true,
    unique: true
},
```

- Password

```
//Password
password: {
    type: String,
    required: true
},
```

- Role

```
//Role
role: {
    type: String,
    enum: ["user", "admin"],
    default: "user"
}
```

- Timestamps

```
{ timestamps: true });
```

Product Schema:

- Name

```
//Name
name: {
    type: String,
    required: true
},
```

- Brand

```
//Brand
brand: {
    type: String,
    required: true
},
```

- Price

```
//Price
price: {
    type: Number,
    required: true
},
```

- Category

```
//Category
category: {
    type: String,
    required: true
},
```

- Description

```
//Description
description: {
    type: String
},
```

- Notes

```
//Notes
topNotes: [String],
middleNotes: [String],
baseNotes: [String],
```

- Image URL

```
//Image URL
imageUrl: {
    type: String
},
```

- Creation date.

```
//Date
createdAt: {
    type: Date,
    default: Date.now
}
```

Order Schema:

- Items

```
//Items
items: [
    {
        productId: {
            type: mongoose.Schema.Types.ObjectId,
            ref: "Product"
        },
        quantity: Number
    }
],
```

- Total amount

```
//Total amount
totalAmount: {
    type: Number,
    required: true
},
```

- Payment status

```
//Payment status
paymentStatus: {
    type: String,
    default: "Pending"
},
```

- Creation date.

```
//Date
createdAt: {
    type: Date,
    default: Date.now
}
```

3.3 Authentication System

3.3.1 User Registration

When a user registers:

- System checks if emails exists.

```
//Check if email exists
const existingUser : Query<...> & ObtainSchemaGeneric<module:mon... = await User.findOne( filter: { email });

if (existingUser) {
    return res.status(400).json({ message: "User already exists" });
}
```

- Password is hashed using bcrypt.(*Password hashing in Node.js with bcrypt*, 2024)

```
//Hash password
const salt : string = await bcrypt.genSalt( rounds: 10);
const hashedPassword : string = await bcrypt.hash(password, salt);
```

- User is stores in database.

```
//Create new user
const user : (... = await User.create( docs: {
    name,
    email,
    password: hashedPassword
});
```

- Default role is assigned as “user”.

3.3.2 Login and JWT

During login:

- Email is verified.

```
//Find user by email
const user : Query<...> & ObtainSchemaGeneric<module:mon... = await User.findOne( filter: { email });

if (!user) {
    return res.status(400).json({ message: "Invalid credentials" });
}
```

- Password hash is performed.

```
//Compare password with stored hash
const isMatch : boolean = await bcrypt.compare(password, user.password);

if (!isMatch) {
    return res.status(400).json({ message: "Invalid credentials" });
}
```

- JWT token is generated.

```
//Create JWT token
const token = jwt.sign(
    { id: user._id, role: user.role },
    JWT_SECRET,
    { expiresIn: "1d" }
);
```

- Token contains: user ID, role and expiration date.

3.3.3 Route Protection

Protected routes use middleware:

- Protect: verifies token.

```
exports.protect = (req, res, next) : any | undefined => {

    //Get token from header
    const token : string = req.headers.authorization;

    if (!token) {
        return res.status(401).json({ message: "Not authorized" });
    }

    try {

        //Verify token
        const decoded : any = jwt.verify(token, JWT_SECRET);

        //Attach user data to request
        req.user = decoded;

        next();

    } catch (error) {
        res.status(401).json({ message: "Token invalid" });
    }
};
```

- AdminOnly: checks role.

```
//Only admin allowed
exports.adminOnly = (req, res, next) : any | undefined  => {

    if (req.user.role !== "admin") {
        return res.status(403).json({ message: "Admin access required" });
    }

    next();
};
```

3.4 Frontend

Each page was designed for a specific functionality:

- Products.html: product listing.

```
<!DOCTYPE html>
<html>
<head>
    <title>Perfume Collection</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<h1>Our Perfume Collection</h1>

<a href="cart.html">View Cart</a>
<a href="orders.html">Order History</a>
<a href="admin.html">Admin Panel</a>

<!-- Search input -->
<input
        type="text"
        id="searchInput"
        placeholder="Search perfume..."
        onkeyup="searchProducts()"
/>

<div id="product-list" class="grid"></div>

<script src="js/app.js"></script>
</body>
</html>
```

- Cart.html: cart management.

```
<!DOCTYPE html>
<html>
<head>
    <title>Your Cart</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<h1>Your Cart</h1>

<a href="products.html">← Back to Products</a>
<br><br>

<div id="cart-items"></div>
<h2 id="total"></h2>

<button onclick="clearCart()">Clear Cart</button>

<hr>

<!--PayPal-->
<script src="https://www.paypal.com/sdk/js?client-id=ASoUgmpYh1ZBHoIW-QI8hhqhjpmX8_2K12ayinamHJyP4DjsTYhA_1UAAtSvSWvOhaYAb2ehUBGKXQVW&currency=EUR"></script>

<div id="paypal-button-container"></div>

<script src="js/cart.js"></script>

</body>
</html>
```

- Orders.html: order history.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Order History</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<h1>Your Orders</h1>

<!-- Orders will appear here -->
<div id="order-list"></div>

<script src="js/orders.js"></script>

</body>
</html>
```

- Admin.html: product management.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Add Product</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<h1>Add New Perfume</h1>

<!-- Product Form -->
<form id="productForm">

    <input type="text" id="name" placeholder="Name" required>
    <input type="text" id="brand" placeholder="Brand" required>
    <input type="number" id="price" placeholder="Price" required>
    <input type="text" id="category" placeholder="Category" required>
    <input type="text" id="imageUrl" placeholder="Image URL">

    <button type="submit">Add Product</button>

</form>

<script src="js/admin.js"></script>

</body>
</html>
```

- Login.html: authentication.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<h1>Login</h1>

<form id="loginForm">
    <input type="email" id="email" placeholder="Email" required>
    <input type="password" id="password" placeholder="Password" required>
    <button type="submit">Login</button>
</form>

<script src="js/auth.js"></script>

</body>
</html>
```

- Register.html: user creation.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Register</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<h1>Register</h1>

<form id="registerForm">

    <!-- Username input -->
    <input type="text" id="name" placeholder="Name" required>
    <!-- Email input -->
    <input type="email" id="email" placeholder="Email" required>
    <!-- Password input -->
    <input type="password" id="password" placeholder="Password" required>
    <!-- Submit button -->
    <button type="submit">Register</button>

</form>

<!-- Link to JS -->
<script src="js/register.js"></script>

</body>
</html>
```

3.4.1 HTML Design

Structure:

- Header section.
- Navigation links.
- Content container.
- Script integration.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Chehab - Luxury Perfumes</title>

    <!-- Link CSS -->
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

<!-- Hero Section -->
<section class="hero">
    <h1>Chehab</h1>
    <p>Discover Your Style!</p>
    <a href="products.html">Shop Now</a>
</section>

</body>
</html>
```

Each page includes:

- A main container.
- Dynamic content placholder.
- Script reference for JS logic.

3.4.2 CSS Styling

Design:

- Dark background.
- Gold accents for buttons.
- Card layout.
- Flexbox grid system.
- Rounded corners

- Soft shadows.

```css
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #111;
    color: white;
    text-align: center;
}
.grid {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 20px;
}
.card {
    background: #1c1c1c;
    padding: 20px;
    border-radius: 10px;
    width: 220px;
    box-shadow: 0 0 10px rgba(255,215,0,0.3);
}
button {
    background: gold;
    border: none;
    padding: 8px 12px;
    cursor: pointer;
}
```

Logic:

- .grid uses flexbox for responsive layout.
- .card encapsulates each product visually.

3.4.3 JavaScript

It is used to:

- Fetch data from backend API

```
const API_URL : string = "/api/products";
```

- Dynamically render products.

```
async function fetchProducts() : Promise<void> { Show usages

    const response : Response = await fetch(API_URL);
    const products = await response.json();

    const productList : HTMLElement = document.getElementById( elementId: "product-list");
    productList.innerHTML = "";

    products.forEach(product => {

        const div : HTMLDivElement = document.createElement( tagName: "div");
        div.className = "card";

        div.innerHTML = `
      <img src="${product.imageUrl || 'https://via.placeholder.com/200'}" width="150">
      <h3>${product.name}</h3>
      <p>${product.brand}</p>
      <p><strong>${product.price} €</strong></p>
      <button onclick="addToCart('${product._id}', '${product.name}', ${product.price})">
        Add to Cart
      </button>
    `;

        productList.appendChild(div);
    });
}
```

- Manage cart state.

```
function addToCart(id, name, price) : void  {  Show usages

    const existing : T  = cart.find(item : T  => item.productId === id);

    if (existing) {
        existing.quantity++;
    } else {
        cart.push({
            productId: id,
            name,
            price,
            quantity: 1
        });
    }

    localStorage.setItem("cart", JSON.stringify(cart));

    alert("Added to cart!");
}
```

- Handle form submission.

```
function searchProducts() : void  {  Show usages

    //Get value typed by user
    const searchValue = document.getElementById( elementId: "searchInput").value;

    //Fetch products using search query
    fetch( input: `/api/products?search=${searchValue}`) Promise<Response>
        .then(res : Response  => res.json()) Promise<any>
        .then(products => {

            const productList : HTMLElement  = document.getElementById( elementId: "product-list");
            productList.innerHTML = "";

            products.forEach(product => {

                const div : HTMLDivElement  = document.createElement( tagName: "div");
                div.className = "card";

                div.innerHTML = `
        <h3>${product.name}</h3>
        <p>${product.brand}</p>
        <p>${product.price} €</p>
        <button onclick="addToCart('${product._id}', '${product.name}', ${product.price})">
          Add to Cart
        </button>
      `;
```

3.5 PayPal Integration

The system uses PayPal Sandbox REST API.
Steps:

- Frontend loads PayPal SDK.(*JavaScript SDK*, no date)
- Backend generates OAuth token.

```
async function getAccessToken() : Promise<any>  {  Show usages

    //Send request to PayPal OAuth endpoint
    const response : AxiosResponse<any>  = await axios({
        url: "https://api-m.sandbox.paypal.com/v1/oauth2/token",
        method: "post",

        //Basic auth using Client ID + Secret
        auth: {
            username: PAYPAL_CLIENT_ID,
            password: PAYPAL_SECRET
        },

        headers: {
            "Content-Type": "application/x-www-form-urlencoded"
        },

        //Required body parameter
        data: "grant_type=client_credentials"
    });

    //Return access token
    return response.data.access_token;
}
```

- Backend creates PayPal order.

```
exports.createPayPalOrder = async (req, res) : Promise<...> => {
    try {
        //Get total from frontend
        const { totalAmount } = req.body;
        //Prevent invalid amounts
        if (!totalAmount || totalAmount <= 0) {
            return res.status(400).json({ message: "Invalid amount" });
        }
        //Get PayPal access token
        const accessToken = await getAccessToken();
        //Create order in PayPal
        const response : AxiosResponse<any> = await axios({
            url: "https://api-m.sandbox.paypal.com/v2/checkout/orders",
            method: "post",
            headers: {
                Authorization: `Bearer ${accessToken}`,
                "Content-Type": "application/json"
            },
            data: {
                intent: "CAPTURE",
                purchase_units: [
                    {
                        amount: {
                            currency_code: "EUR",
                            value: totalAmount.toString()
                        }
                    }
                ]
            }
        });
```

- User approves payment.
- Backend captures payment.

```
exports.capturePayPalOrder = async (req, res) : Promise<void> => {

    try {
        const { orderId, cartItems, totalAmount } = req.body;

        //Get PayPal access token
        const accessToken = await getAccessToken();
        //Capture the approved order
        await axios({
            url: `https://api-m.sandbox.paypal.com/v2/checkout/orders/${orderId}/capture`,
            method: "post",
            headers: {
                Authorization: `Bearer ${accessToken}`,
                "Content-Type": "application/json"
            }
        });
```

- Order stored in MongoDB.

```
//Save order to MongoDB after successful capture
const newOrder : (... = await Order.create( docs: {
    items: cartItems,
    totalAmount,
    paymentStatus: "Completed"
});

res.json({
    message: "Payment successful",
    order: newOrder
});
```

3.6 Docker Integration

MongoDB runs inside docker container for macOS 12, this ensured:

- Isolated database.
- No local installation.
- Easy replication on another device.

Command used:

docker run -d -p 27017:27017 --name perfume-mongo mongo:7

4. Result Analysis

4.1 Overview

After the implementation of the frontend, backend, database, authentication system and PayPal payment, the system successfully showed:

- User registration and login.
- Product browsing and search.
- Cart management.
- Secure payment process.
- Order history.
- Admin only product creation.

4.2 Steps

- Run MongoDB with Docker

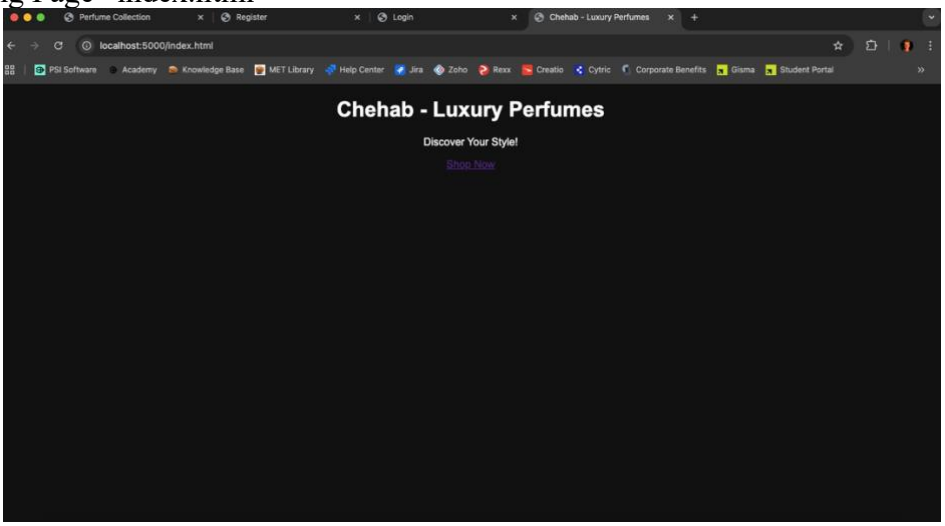
```
chippo@Chippos-MacBook-Air Chehab Perfume Store % docker run -d -p 27017:27017 --name perfume-mongo mongo:7
11084a36b0db4b999b927d6297b51f59d8eb6331c0d8347f64ed685ba642acd9
```

- Start Server

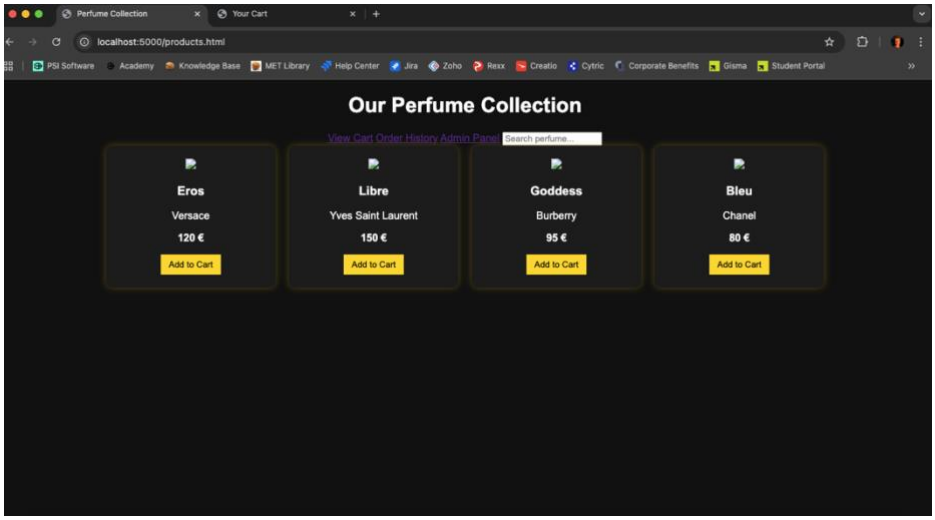
```
chippo@Chippos-MacBook-Air Chehab Perfume Store % npm run dev

> perfume-store@1.0.0 dev
> nodemon server/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server/server.js`
Server running on port 5000
MongoDB Connected
```

4.3 Page Screeshots

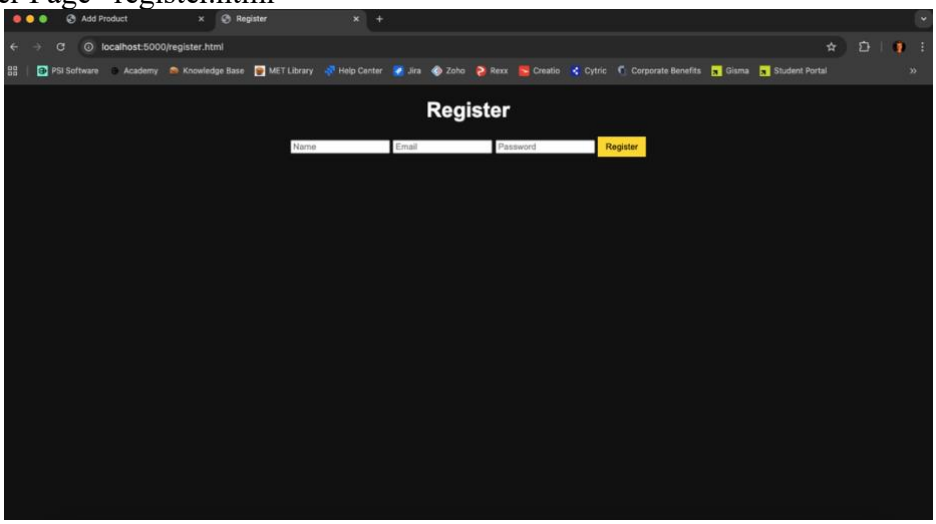
- Landing Page “index.html”



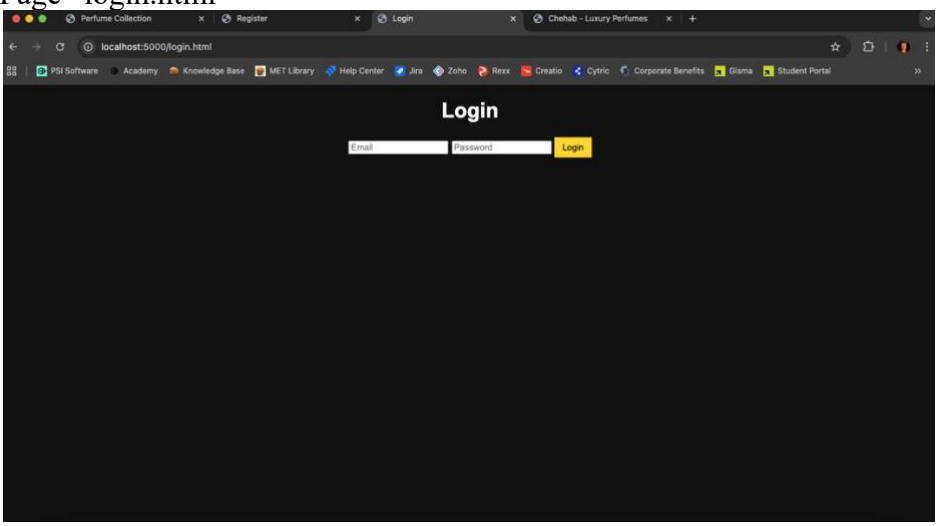
- Products Page “products.html”



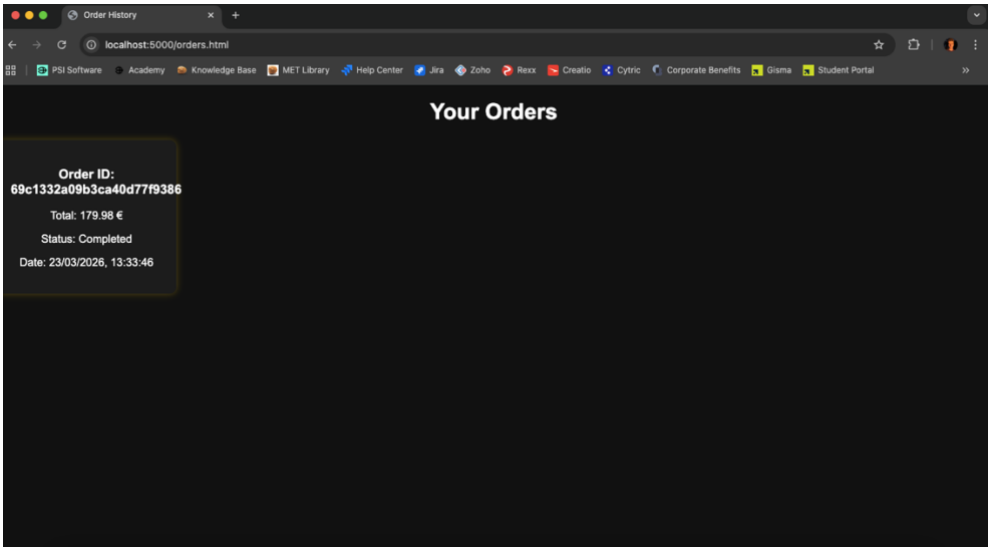
- Register Page “register.html”



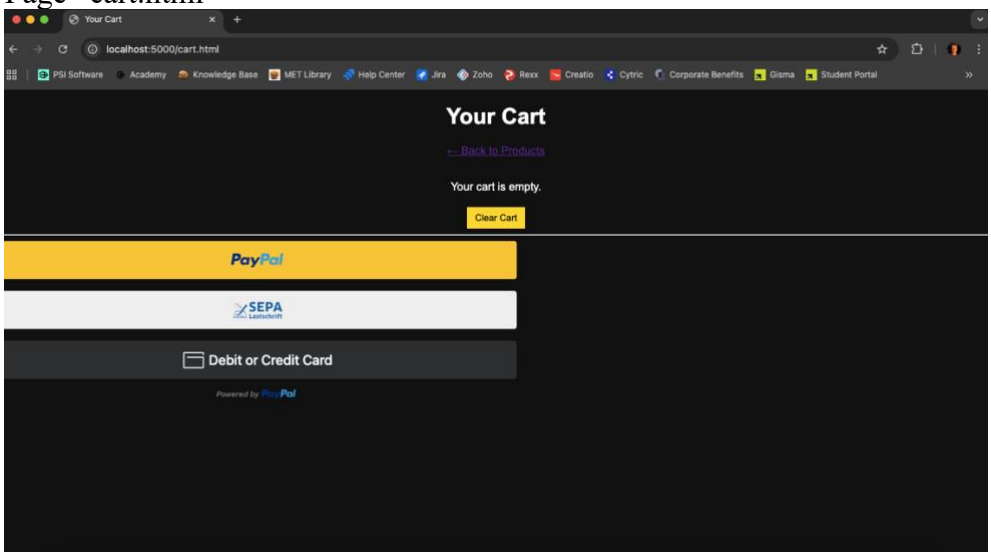
- Login Page “login.html”



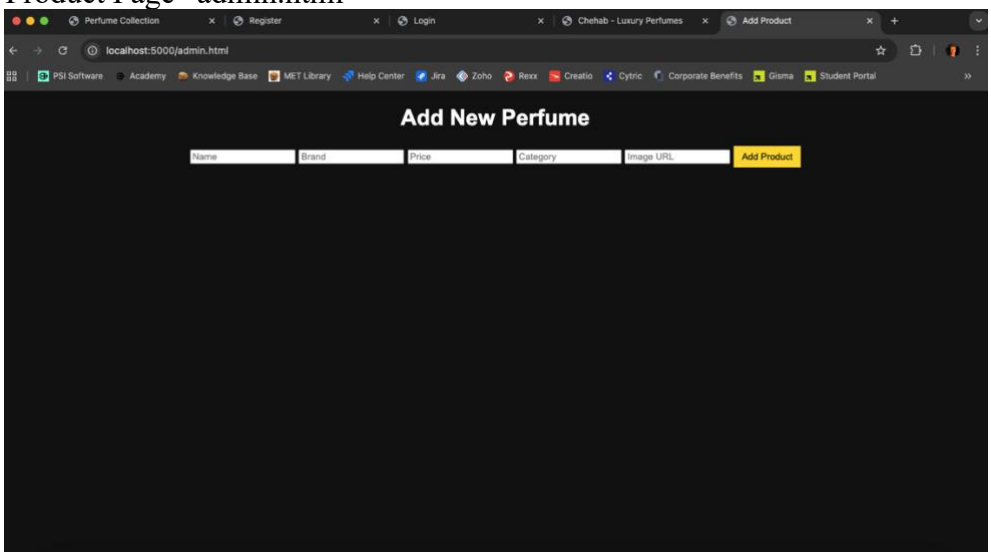
- Order History Page “order.html”



- Cart Page “cart.html”



- Add Product Page “admin.html”

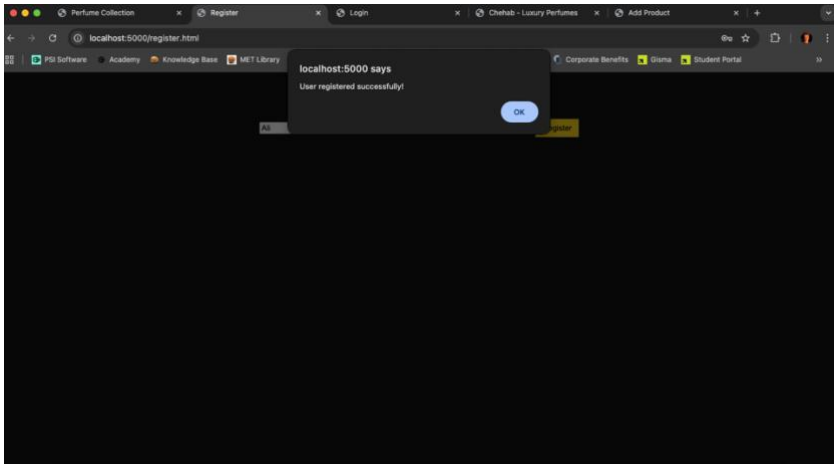


4.4 User Authentication

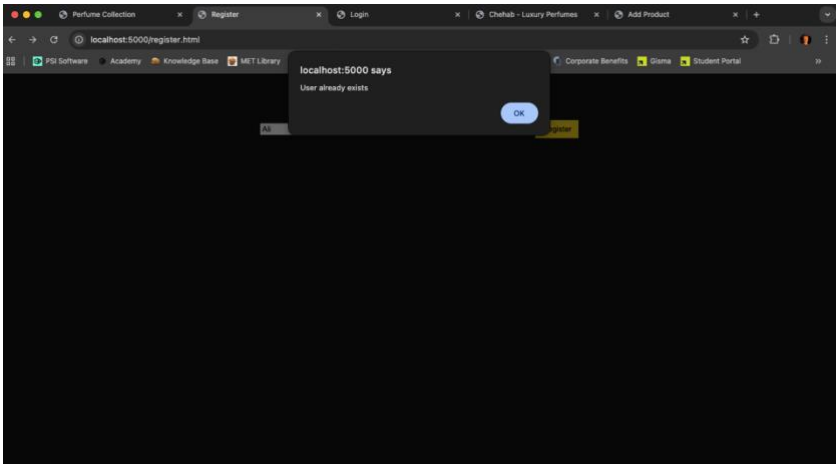
Test cases applied:

S/N	Test Case	Expected Result	Actual Result
1	Register new user	Account created	Successful
2	Register existing user	Error message	Successful
3	Login with correct credentials	JWT issued	Successful
4	Login with wrong credentials	Error returned	Successful
5	Add product with admin access	Product added	Successful

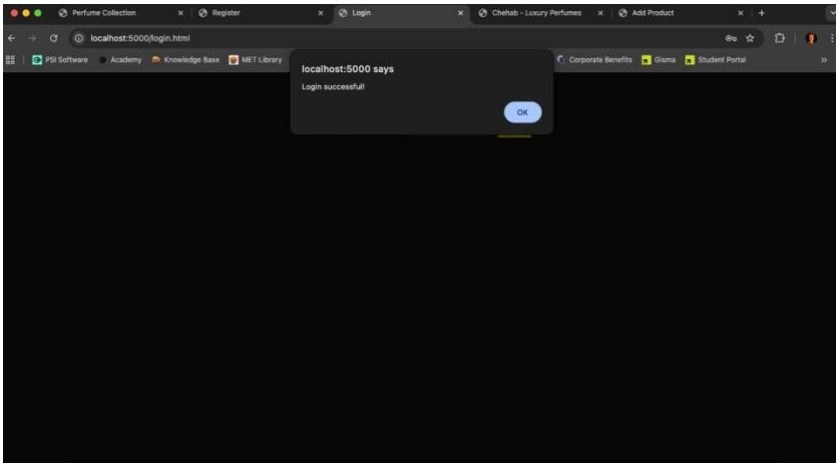
- Test Case #1:



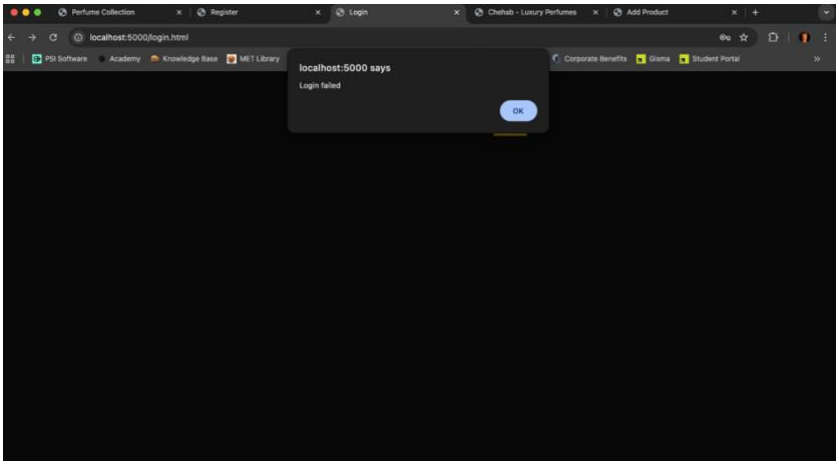
- Test Case #2:



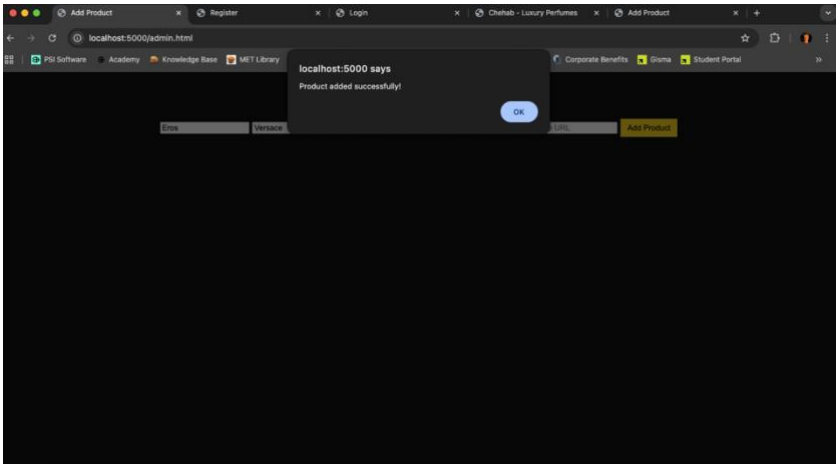
- Test Case #3:



- Test Case #4:



- Test Case #5:



4.5 Product Management

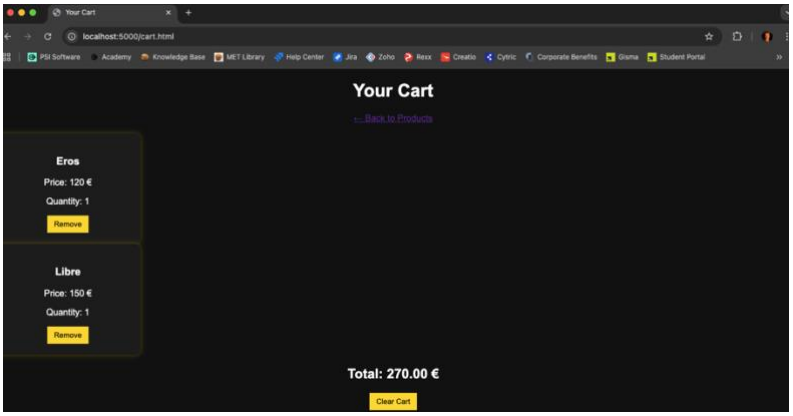
The product module has successfully showed:-

- Adding new products.
- Searching for products.
- Fetching all products.
- Browising all products.

4.6 Cart Management

Cart operations has been tested for:

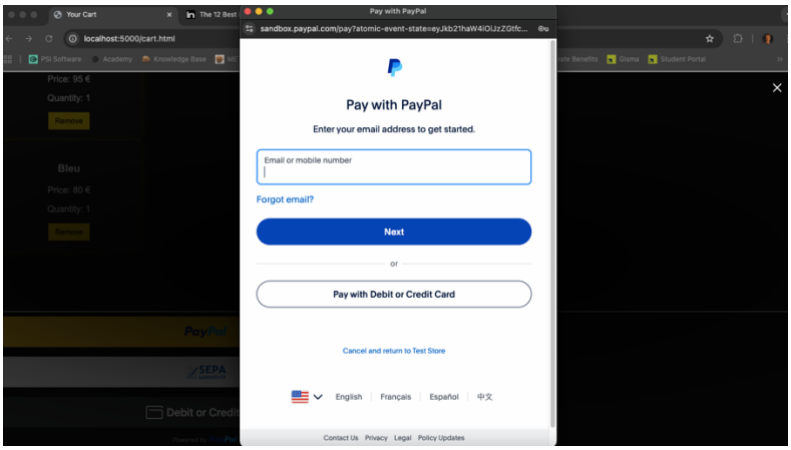
- Adding items.
- Increasing quantity.
- Removing items.
- Clearing items.



4.7 PayPal Payment

The PayPal Sandbox integration was tested using:

- Valid Sandbox personal account.
- Multiple transactions amount.
- Emty cart scenario.
- Payment rejection scenario.



4.8 Summary

The authentication system worked as expected. Password hashing using bcrypt ensured secure storage, while JWT-based authentication allowed stateless session management. Product data was correctly retrieved and rendered dynamically using JavaScript. Since cart data is stored locally, it reduces server load and improves responsiveness.

The project successfully simulates a real-world e-commerce architecture using modern web development technologies.

5. Challenges and Solutions

5.1 Challenge #1

- **Description:** MongoDB connection error on my Apple macOS 12.
- **Solution:** Used Docker container.

5.2 Challenge #2

- **Description**: JWT protection not working.

- **Solution**: Implemented middleware correctly.

5.3 Challenge #3

- **Description**: PayPal payment not complete.
- **Solution**: Fixed credentials problem as per Sandbox.

5.4 Challenge #4

- **Description**: Admin was user role and couldn't add product.
- **Solution**: Implemented admin role in middleware.

5.5 Challenge #5

- **Description**: Client ID and Secret ID were public.
- **Solution**: Moved both to .env file.

6. Conclusion

Finally, the project followed all the requirments given and demonstrated the implementation of a full stack web application using the recent technologies. The implementation of desiging simple webpage, authentication, database management and payment processing showed a basic and simple backend architecture.

7. Future Work

- Order tracking status.
- Cloud storage.
- User account management
- Product categories.
- Email notifications.
- "Contact Us" form.
- React/Vue implementation.

---

8. References

Alwis, L.W. (2025) “Implementing Microsoft OAuth2 Login in Node.js (TypeScript + Express),” *Medium*, 24 October. Available at: https://masterlwa.medium.com/implementing-microsoft-oauth2-login-in-node-js-typescript-express-ebc59a88aacb (Accessed: March 11, 2026).

Controllers in ExpressJS (14:43:00+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/web-tech/controllers-in-expressjs/ (Accessed: March 12, 2026).

Express routing (no date). Available at: https://expressjs.com/en/guide/routing.html (Accessed: March 12, 2026).

Get Started with PayPal REST APIs (no date) *PayPal API reference*. Available at: https://developer.paypal.com/api/rest/ (Accessed: March 11, 2026).

JavaScript SDK (no date). Available at: https://developer.paypal.com/sdk/js/ (Accessed: March 12, 2026).

JSON Web Token (JWT) (00:00:39+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/web-tech/json-web-token-jwt/ (Accessed: March 20, 2026).

Middleware in Express (00:30:07+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/node-js/middleware-in-express-js/ (Accessed: March 20, 2026).

Mongoose Tutorial (16:52:58+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/node-js/mongoose-tutorial/ (Accessed: March 11, 2026).

Node.js — How to read environment variables from Node.js (no date). Available at: https://nodejs.org/en/learn/command-line/how-to-read-environment-variables-from-nodejs (Accessed: March 18, 2026).

Node.js Middleware Concept (no date). Available at: https://www.w3schools.com/nodejs/nodejs_middleware.asp (Accessed: March 14, 2026).

Node.js REST API (no date). Available at: https://www.w3schools.com/nodejs/nodejs_rest_api.asp (Accessed: March 17, 2026).

Password hashing in Node.js with bcrypt (2024) *Honeybadger*. Available at: https://www.honeybadger.io/blog/node-password-hashing/ (Accessed: March 11, 2026).

Running MongoDB in Docker: A Complete Guide with Examples (no date). Available at: https://www.datacamp.com/tutorial/running-mongodb-in-docker?utm_cid=19589720821&utm_aid=186331394549&utm_campaign=230119_1-ps-other~dsa-tofu~all_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9194002-&utm_mtd=-c&utm_kw=&utm_source=google&utm_medium=paid_search&utm_content=ps-other~emea-en~dsa~tofu~tutorial~mongodb&gad_source=1&gad_campaignid=19589720821&gbraid=0AAAAADQ9WsFrzAqQO0KGnfBVPIN7HlC1W&gclid=Cj0KCQjwj47OBhCmARIsAF5w

UEH_hjQvuwZSmrEOXDs1Ba-i07EzPwvAva2mAfRVpFx-7aiwmj75x4QaAi65EALw_wcB (Accessed: March 21, 2026).